# DATEVRA — COMPLETE TINDER-STYLE USER EXPERIENCE UPGRADE

*Five-phase comprehensive master implementation prompt | Existing React + Supabase application*

## IMPORTANT: EXECUTE THIS AS ONE CONTROLLED MASTER PROJECT

Implement the complete five-phase user-facing dating experience upgrade in the existing Datevra application. The user wants Tinder-like functionality and user experience across account creation, onboarding, profiles, discovery, likes, matching, compatibility, chat and notifications. This is one coordinated project, not five disconnected redesigns. Audit first, then implement sequentially in the same workspace, preserving working features and existing user data.

## 1. NON-NEGOTIABLE PROJECT RULES

- Keep Datevra's existing name, logo, luxury visual identity, colours, typography, gradients, components, spacing system and brand language. Use Tinder only as a reference for interaction patterns and feature concepts. Never copy Tinder's name, logo, exact screens, wording, colours, artwork or proprietary visual design.
- Do not redesign or modify the Admin Panel. Scope this project to member-facing account creation/onboarding, profile, discovery, likes, matches, chat and related member navigation.
- Do not break existing accounts, authentication, profile records, photos, likes, matches, messages, subscriptions, passes, payments or other existing data. Preserve backward compatibility.
- The project already has some Tinder-style features (approximately half, according to the project owner). Do not assume features are missing. Audit the actual code, Supabase schema, RLS policies, RPCs, edge functions and current UI before changing anything. Reuse and improve working components and handlers; do not create duplicate systems.
- No fake/demo users, fake matches, fake likes, fabricated profile answers, simulated persistence or success messages that are not confirmed by the real application. Do not seed production with sample users.
- Use the existing Supabase project and established data architecture. Persist real user data through authenticated, authorized operations. Never expose service-role keys, secrets or private profile data in client code.
- Do not weaken RLS, authorization, account ownership checks, age/privacy restrictions or existing access rules. A user must not be able to read or modify another user's private data or send messages outside permitted match rules.
- Do not execute SQL, apply migrations, alter production database records, change production settings, environment secrets, DNS, hosting or deployment configuration. If schema changes are required, create a clearly named migration file in the local workspace and explain the exact reason, impact and safe application steps. Do not run it.
- Do not commit, push, publish or deploy. Work only in the current Google AI Studio workspace.
- Do not add dependencies unless genuinely necessary. Prefer current project libraries, components, design tokens and patterns.
- Do not remove or rewrite existing features simply because a new UI is easier to build. Keep routes and deep links compatible.

## 2. MANDATORY INITIAL AUDIT — BEFORE EDITING

Map the current app routes and member journey from registration through onboarding, profile completion, discovery, likes, matches and chat.

Inspect authentication flows, profile tables/types/mappers, photo storage, discovery queries, swipe/like/pass handlers, match creation, compatibility logic, chat/message persistence, notifications and realtime subscriptions.

Inspect Supabase RLS policies and existing RPC/Edge Function contracts relevant to these features. Read code and schema artifacts only; do not execute SQL or change remote state.

Create an internal feature matrix: already working; partially working; UI-only; broken; missing; and blocked by schema/backend dependency.

Identify existing users' profile fields and preserve all existing data. Plan safe defaults for legacy accounts so existing members are not forced through registration again.

Only after this audit, implement the missing or incomplete parts. Do not replace a working feature with a new parallel implementation.

## PHASE 1 — ACCOUNT CREATION, STEP-BY-STEP ONBOARDING AND PROFILE SETUP

Build a guided, multi-step onboarding journey inspired by the clarity and ease of Tinder's onboarding, while using Datevra's own visual design. Keep registration and profile setup understandable, mobile-first and resumable.

### A. Account creation and verification

- Retain the existing Datevra authentication provider and secure signup/login flow. Collect only fields required by the existing product and legal requirements.
- Normalize email consistently and handle duplicate email registrations with a clear account-already-exists message and links to Login and Forgot Password. Do not reveal whether an email belongs to an account through insecure unauthenticated APIs; follow Supabase's

secure auth behavior.

- Keep email confirmation and password reset working. Never bypass verification or weaken authentication.
- After successful registration, create or retrieve the authenticated user's own profile safely and resume onboarding at the first incomplete step.

### B. Onboarding steps and profile questions

Basic identity: name, date of birth/age as supported by the existing schema, gender and location. Validate required fields and apply the product's existing age eligibility rules. Do not infer age or identity.

About the user: short bio/introduction, education and occupation only where these fields already exist or are approved by the product. Make optional fields clearly optional.

Interests and hobbies: allow selection from the existing supported interests/tags. If no controlled list exists, inspect current data conventions before adding one; do not silently invent a taxonomy or break existing records.

Lifestyle and personality: present only questions and answer choices supported by existing Datevra fields or approved requirements. Make answers editable and store them against the authenticated user's profile.

Dating intention: ask the user's relationship/dating intention and relevant preferences only where the existing product supports them. Avoid making any preference mandatory unless the current product requires it.

Partner preferences: collect existing supported discovery preferences such as preferred age range, distance/location or other currently implemented criteria. Clearly explain optional preferences and validate ranges.

Photos: use the existing secure upload/storage path. Allow the user to select, preview, reorder or remove photos only if existing upload and authorization flows support those operations. Validate file type/size, handle upload errors, and never expose private storage paths or fabricate images.

Review: show a clear summary before completion, let the user go back and edit answers, and submit only the fields the user has actually supplied.

- Show step progress, back/continue controls, inline validation and clear error messages. Preserve entered values when moving between steps or after recoverable errors.
- Allow users to leave and resume onboarding without losing completed answers. Do not block existing fully registered members with a new mandatory onboarding flow.
- Create a profile completion indicator only if it can be calculated from real persisted fields. Do not invent completion percentages.

### C. Profile editing

- Provide a member-facing edit profile experience using the same fields and validations as onboarding. Users can update their own profile only.
- Preserve existing profile values not included in an edit request. Avoid accidental nulling or overwriting of fields.

## PHASE 2 — DISCOVERY, PROFILE CARDS AND SWIPE EXPERIENCE

- Improve the existing discovery route; do not create a second discovery system. Display eligible real profiles returned by existing authorized discovery logic.
- Create a clean, visually compelling profile-card hierarchy using Datevra's theme. Show only essential discovery information: available profile photo, name, age if permitted, location and a short introduction or selected highlights.
- Move lengthy biographies, detailed questionnaire answers, extensive interests and secondary information into Full Profile. This follows the client's explicit instruction: do not show long details on the discovery card; show them when the user clicks Full Profile.
- Support intuitive left/right swipe gestures on touch devices and equivalent accessible Like/Pass controls on desktop. Prevent accidental page navigation and preserve normal page scrolling outside deliberate card gestures.
- Add subtle card transitions and action feedback. Respect prefers-reduced-motion. Prevent double taps/submissions while an action is processing.
- Connect Like and Pass to the existing real action handlers. Do not show success until the backend confirms the operation. Handle network errors and allow safe retry without duplicate records.
- If enhanced actions such as Super Like already exist, expose them correctly according to actual entitlement rules. Do not invent quotas, paid benefits or purchase flows.
- Support existing multi-photo profiles with clear progress indicators and tap/swipe photo navigation. Preserve image order and current storage conventions.
- Improve existing filters only where backed by actual query logic. A filter must not appear active unless it is applied to the real discovery query.
- Include real loading, no-more-profiles, no-results, offline and error states. Never substitute demo profiles when real results are empty.

## PHASE 3 — LIKES, MUTUAL MATCHES AND COMPATIBILITY

- Audit existing like/pass records and match creation logic. Reuse current tables, unique constraints and server-side protections.
- Implement or complete the Likes experience using real incoming/outgoing like data and existing privacy rules. Do not reveal private incoming likes if the current product's entitlement/access rules do not permit it.
- Create a match only when the existing product's mutual-like rules are satisfied. Enforce duplicate prevention and idempotency so retries or simultaneous likes cannot create duplicate matches.
- Show real match confirmation and match list entries only after confirmed persistence. Ensure unmatch/block/report behavior, if already supported, continues to work.
- Use existing profile answers and preferences for compatibility. First inspect whether a compatibility score or ranking model already exists. Reuse it if present; do not claim scientific certainty or invent compatibility percentages.
- If compatibility scoring is missing, do not fabricate a score. Implement only a transparent, explainable matching/ranking approach supported by existing data and product rules. If new backend logic/schema is required, document it and create local migration artifacts only; do not execute them.
- Respect blocked users, hidden/deleted accounts, verification/access restrictions and discovery exclusions already defined by Datevra.

## PHASE 4 — MATCHED CHAT, NOTIFICATIONS AND MEMBER NAVIGATION

- Use the existing chat and message system. Only matched/authorized members may communicate where that is the current Datevra rule.
- Improve conversation list, message view, unread indicators, timestamps, sending state, failed-send retry and empty states without changing the underlying message contract unnecessarily.
- Use existing Supabase realtime or notification infrastructure if present. Do not create duplicate subscriptions, duplicate notifications or uncontrolled polling.
- Show notifications for real likes, matches and messages only where the current product rules permit. Prevent repeated notifications for the same event and respect user settings.
- Improve member navigation among Discover, Likes, Matches, Chat and Profile using the existing routes. Keep mobile navigation touch-friendly and desktop navigation consistent.
- Preserve message history and existing conversations. Do not delete, migrate or rewrite historical messages.

## PHASE 5 — RESPONSIVENESS, ACCESSIBILITY, SECURITY AND END-TO-END QUALITY

- Make all changed member-facing screens work at narrow mobile, tablet and desktop widths. Prevent horizontal overflow, clipped text, inaccessible dialogs and controls hidden behind safe areas.
- Use comfortable touch targets (approximately 44px minimum where practical), readable typography, keyboard navigation, semantic controls, visible focus states and accessible labels.
- Ensure dialogs/full-profile panels can be opened, closed, scrolled and navigated correctly on mobile and desktop. Restore focus sensibly when dialogs close.
- Optimise image loading and rendering using the existing image pipeline. Avoid loading every large image at once where the existing architecture supports lazy loading.
- Check performance for unnecessary rerenders, duplicate requests, unbounded realtime listeners, memory leaks and excessive polling. Do not change hosting or deployment configuration.
- Test authentication, onboarding resume, profile editing, photo actions, discovery, filters, Like, Pass, mutual match, chat, notifications and navigation with existing test infrastructure.
- Test failure cases: duplicate signup, expired session, unauthorized profile edit, blocked user, network failure, duplicate action, empty discovery, image upload failure and realtime reconnect.
- Run TypeScript checks, lint, relevant automated tests and production build. Fix regressions introduced by this work.

## 3. DATA AND SUPABASE CHANGE CONTROL

Before changing data access, document the current table/column/RPC/function/policy used by each feature. Prefer existing schema and contracts. If a required field, unique constraint, RPC or policy is absent, do not guess its intended semantics.

- Never use the Supabase service-role key in browser code. Use authenticated user context and existing server-side trusted operations.
- Never weaken RLS to make the UI work. Ensure profile reads expose only approved fields and writes are limited to the owner or authorized admin workflows.
- Do not store sensitive onboarding answers in public profile fields unless the current product explicitly requires that visibility.
- For any required schema change, add a local migration file with comments, rollback considerations and a clear explanation. Do not execute it or apply it remotely. Mark dependent features as blocked until the project owner approves and applies the migration.

## 4. IMPLEMENTATION ORDER AND WORKING METHOD

Complete the audit and feature matrix first.

Implement Phase 1, then Phase 2, then Phase 3, Phase 4 and Phase 5 in order, all within this one master task and current workspace.

After each phase, run focused checks and inspect the changed code before continuing. Do not abandon earlier working features while adding later phases.

Keep edits narrow and explain why each touched file is necessary. Avoid broad refactors and unrelated formatting churn.

If a phase is blocked by missing schema/backend support, finish all safe UI/code work that can be completed without faking functionality, create local migration artifacts if needed, and clearly identify the exact blocker. Do not silently skip it or pretend it is complete.

## 5. FINAL ACCEPTANCE CHECKLIST

- A new user can register, verify their account and complete the supported step-by-step onboarding without losing answers.
- An existing user can log in without being forced to recreate their account or lose existing profile data.
- A member can edit their own profile and supported photos safely.
- Discovery displays real eligible members, with concise cards and detailed Full Profile views.
- Swipe gestures and Like/Pass controls invoke real existing actions and handle errors safely.
- Likes and mutual matches reflect persisted backend state and do not create duplicates.
- Compatibility information is based on real supported data and is not fabricated.
- Chat and notifications use existing authorized infrastructure and preserve message history.
- Member navigation and all changed screens work on mobile, tablet and desktop.
- Datevra branding and Admin Panel remain unchanged.
- No fake data, fake persistence, unauthorized data exposure, SQL execution, remote migrations, production changes, commits, pushes or deployments occurred.

## 6. REQUIRED FINAL IMPLEMENTATION REPORT

Return a precise report with: (1) audit findings and feature matrix; (2) each phase completed/partially completed/blocked; (3) exact files created or modified and why; (4) existing Supabase tables/RPCs/functions reused (names only, no secrets); (5) any local migration files created and why; (6) which interactions were verified against real backend handlers; (7) tests, lint, TypeScript and build results; (8) mobile/desktop preview checks; (9) known bugs or blockers; and (10) explicit confirmation that no SQL was executed, no remote database or production data was changed, and no commit/push/deployment occurred.

**COMPLETION RULE: Do not claim “Tinder-like experience complete” merely because the UI looks similar. Report each feature honestly as working, partial, blocked or not implemented. The project is complete only when the supported user journey works end-to-end with real authenticated Supabase persistence, existing security rules and verified responsive UI. Preserve Datevra's identity throughout.**